

A.Y. 2025–2026

Software Engineering

for

AstraLog-HPC Project

RASD & DD

Antonello Anzalone
Gabriele Amodeo
Francesco Agosta

Version 1.0
April 2026

Contents

1	The project and project goals	2
1.1	Purpose	2
1.2	Scope	2
1.3	Definitions/Acronyms	2
2	Requirement analysis	4
2.1	Relevant human and non-human actors	5
2.1.1	Human Actors	5
2.1.2	Non Human Actors	5
2.2	Scenarios and Use Cases	5
2.2.1	Scenarios	5
2.2.2	Use Cases	6
2.3	Domain assumptions	9
2.3.1	Configuration and Metadata	9
2.3.2	Data Nature and Temporal Logic	9
2.3.3	Execution Environment (HPC)	9
2.3.4	Universal Standards	10
2.4	Requirements	10
2.4.1	Functional requirements	10
	System Configuration & Initialization	10
	Telemetry Ingestion & Validation	10
	Monitoring Rule Evaluation	10
	Result Reporting & Logging	11
2.4.2	Non functional requirements	11
3	Design	12
3.1	General Description of the Architecture	12
3.1.1	Architectural Components	12
3.1.2	Architectural Connectors	13
3.2	Sequence diagrams	14
3.2.1	Simple & Stateful rule evaluation	14
3.2.2	Logical Correlation Rule Evaluation	15
3.3	Critical Points and Design Decisions	15
3.3.1	Maximizing Performance via Divide and Conquer	15
3.3.2	Mitigating the Limitations of Sharding	16
3.3.3	Modularity, Cohesion, and Defensive Boundaries	16
3.3.4	Information Hiding and I/O Bottleneck Management:	16
3.4	Deployment Diagram	17
4	Work distribution and effort	18
4.1	Personal Contribution and Role in Requirements Analysis and System Design	18
5	Usage of AI	19

1 The project and project goals

1.1 Purpose

The project serves two main purposes. Academically, the objective is to practice software engineering concepts, including requirement engineering, architectural design, automated testing, containerization and the use of High-Performance Computing (HPC) clusters via SLURM. The goal is focused on "learning by doing" rather than just delivering working software.

In the context of the simulated scenario, the purpose is to respond to a "Call for Tenders" from the European Space Agency (ESA) by developing AstraLog-HPC, a software system designed to ensure spacecraft safety by identifying anomalies in aircraft subsystems.

1.2 Scope

The overall project is divided into two phases: creating a requirement analysis and design document (Phase 1), and building a Continuous Integration/Continuous Deployment (CI/CD) pipeline alongside either a provided implementation or a custom prototype built in C++ or Python (Phase 2).

The specific scope of the AstraLog-HPC software system includes:

- **Data Ingestion and Filtering:** Ingesting telemetry data batches from provided `.csv` files and filtering out corrupted data packets caused by transmission noise, specifically targeting malformed JSON, missing mandatory fields (schema errors) and invalid data types.
- **Rule Evaluation:** Accumulating valid packets into batches and applying four types of monitoring rules (Simple Absolute Threshold, Step Difference, Stateful Measurement Persistence, and Logical Correlation) defined in a `rules.json` file. This evaluation relies on a unified synchronous system clock where all measurements are equally spaced in time and perfectly aligned.
- **System Outputs:** The software must strictly adhere to specified output formats, appending nominal states to `valid_data.csv` and anomalous states (rule violations) to `alarms.log`.
- **Deployment Environment:** The system must be capable of running on classical machines/virtual machines as well as on an HPC cluster utilizing the SLURM job scheduler and Singularity containers.

1.3 Definitions/Acronyms

The key definitions and acronyms relevant to the project:

- **HPC:** High-Performance Computing.
- **ESA:** European Space Agency.

- **SLURM**: The job scheduler used to manage and execute jobs on the HPC cluster.
- **CI/CD**: Continuous Integration/Continuous Deployment. The automated pipelines required for building, testing, and releasing the project.
- **JSON**: JavaScript Object Notation. The format in which the spacecraft originally transmits data packets, which are provided to our system as text strings enclosed within static .csv batch files.
- **RASD**: Requirement Analysis and Specification Document. The culmination of the Requirement Engineering process containing the system's purpose, scope, use cases, models, and definitions.
- **DD**: Design Document. The artifact that captures the system architecture and detailed design decisions, including components, interfaces, data structures and deployment considerations, serving as a blueprint for implementation.
- **GenAI / LLMs**: Generative AI / Large Language Models (such as ChatGPT, Gemini and Claude), which can be used to assist the project under strict accountability and ethical guidelines.

2 Requirement analysis

The Machine (AstraLog-HPC)

This is the software-to-be developed to identify anomalies in aircraft subsystems. It includes the logic to receive streamed data, filter invalid packets, and evaluate four types of monitoring rules (simple, step difference, stateful, and correlation). It also encompasses the execution environment, such as the HPC cluster (Galileo 100) and the SLURM job scheduler.

The World (Environment)

This captures the part of the real world influenced by the Machine—namely, the aircraft subsystems under monitoring. Within this context, actors naturally emerge as the external entities that interact with AstraLog-HPC to accomplish specific objectives.

Classification of Phenomena

World Phenomena

These are events occurring entirely outside the system, such as the actual physical conditions of the aircraft hardware and electromagnetic interference that causes real-world data corruption, which the machine cannot observe directly.

Machine Phenomena

These are purely internal software events, including syntactic validation of incoming JSON packets, aggregation of data into local batches, execution of rule logic (simple, stateful, step-based, and correlation rules) and management of internal memory to track sequences of rule violations.

Shared Phenomena

These are events and data residing at the boundary between the environment (World) and the software (Machine).

- **World-Controlled**

- **Telemetry Stream:** A simulated sequence of JSON packets (timestamps, sensor IDs, values) representing spacecraft data, which our system ingests as static `.csv` batch files rather than a continuous live broadcast.
- **Configuration Inputs:** Defined sensor types and monitoring rules provided via `sensors.yaml` and `rules.json`.
- **HPC Scheduling:** Resource allocation and execution timing decided by the SLURM scheduler on the Galileo100 cluster.

- **Machine-Controlled**

- **System Outputs:** Generation of aggregated nominal data in `valid_data.csv` and specific anomaly entries in `alarms.log`.
- **Job Submission:** Initiation of the `job.sh` script and deployment of Singularity containers to the cluster environment.
- **Output Redirection:** Automated capturing of standard output and error logs into dedicated text files.

2.1 Relevant human and non-human actors

2.1.1 Human Actors

Ground Station Engineer: This single actor handles both system setup and ongoing monitoring. His responsibilities include:

- **Configuration:** Defining the types and instances of sensors in the `sensors.yaml` file, and setting the operational parameters—such as batch size—and the monitoring parameters (Simple Absolute Threshold, Step Difference, Stateful Measurement Persistence, and Logical Correlation rules) in the `rules.json` file.
- **Monitoring:** Reviewing the aggregated healthy spacecraft states in the `valid_data.csv` file and responding to rule violations and anomaly alerts generated in the `alarms.log` file.

2.1.2 Non Human Actors

Spacecraft / Sensors: The original external entities in the domain that continuously produce telemetry data (such as temperature, pressure, or voltage) as JSON packets simulated and provided through static `.csv` input files.

2.2 Scenarios and Use Cases

2.2.1 Scenarios

System Initialization

Scenario 1: *Successful System Configuration*

The Ground Station Engineer prepares the system by providing valid `sensors.yaml` and `rules.json` files. They launch AstraLog-HPC and specify a batch size of 100 packets. The software successfully parses the configuration, maps the sensor instances to their respective monitoring rules, and enters a "Ready" state to begin processing the provided telemetry files.

Telemetry Ingestion and Filtering

Scenario 2: *Data Ingestion from CSV Files*

The system reads telemetry data from the provided `.csv` files. AstraLog-HPC parses the records, extracting the timestamp, sensor ID, and numerical values for each aircraft subsystem. The software confirms each record matches the expected schema and appends the data to a local batch for analysis until the user-specified batch size is reached.

Scenario 3: *Filtering Corrupted Telemetry*

While processing the input data, the system encounters records with malformed syntax, missing mandatory fields, or type errors (e.g., a string where a number is expected). AstraLog-HPC's ingestion logic identifies these structural issues, discards the corrupted records, and continues processing the remaining valid data in the file.

Analysis and Outcome Reporting

Scenario 4: *Reporting a Healthy Spacecraft*

After accumulating a full batch, AstraLog-HPC evaluates all monitoring rules and determines that every measurement is within safety limits. The software appends a single aggregated line to `valid_data.csv`, documenting the timestamp and a `NOMINAL` status for all sensors. The Ground Station Engineer reviews the file to confirm the spacecraft's healthy state.

Scenario 5: *Alerting on a Simple Threshold Anomaly*

During rule evaluation, AstraLog-HPC processes a Simple Rule checking if a sensor exceeds a fixed threshold (e.g., `TEMP-01 > 50.0°C`). The system identifies a violation and logs a `MEDIUM` priority alert in `alarms.log`, detailing the timestamp, rule ID, and the violated sensor value.

Scenario 6: *Alerting on a Step-Difference Anomaly*

The system detects that a sensor value has changed by more than the allowed limit compared to the immediately preceding valid measurement. This violates a Step Difference Rule. The system immediately appends an anomaly entry to `alarms.log` for the Ground Station Engineer to review.

Scenario 7: *Monitoring Persistent Voltage Failure*

The software identifies a voltage drop below the required threshold. It monitors the violation through its internal tracking logic by evaluating the number of consecutive valid measurements in which the condition persists. On the final required continuous violation, the system triggers a Stateful Rule and logs the persistent anomaly in `alarms.log`.

Scenario 8: *Identifying Compound Risks*

At a single timestamp, the system identifies that both a high-temperature rule and a pressure-drop rule have been evaluated to be true. AstraLog-HPC evaluates a Logical Correlation Rule with an AND operator. Recognizing the combined risk, the system logs a high-priority alarm in `alarms.log` listing all involved sensors and values.

2.2.2 Use Cases

Use Case 1: System Configuration and Initialization

Name: *Configure and Launch Monitoring*

Participating Actors: Ground Station Engineer

Entry Condition: The `sensors.yaml` and `rules.json` files are present in the designated configuration directory.

Flow of Events:

1. The Ground Station Engineer accesses the AstraLog-HPC command-line interface.

2. The Engineer provides the file paths for the sensor definitions (`sensors.yaml`) and the monitoring rules (`rules.json`).
3. The Engineer specifies the target batch size for telemetry processing.
4. The System parses the configuration files to map specific sensor instances to their allowed types and priorities.
5. The System establishes a connection to the telemetry data source.

Exit Condition: The System enters a "Ready" state and begins ingesting the telemetry data batch.

Special Requirements: The system must provide feedback confirming that all rules have been successfully loaded before starting ingestion.

Use Case 2: Telemetry Ingestion and Validation

Name: *Ingest and Validate Telemetry*

Participating Actors: Spacecraft/Sensor

Entry Condition: AstraLog-HPC is in a "Ready" state and the connection to the data source is active.

Flow of Events:

1. The System reads consecutive telemetry data rows (containing JSON strings) from the provided `.csv` input file.
2. The System parses each packet and aligns the telemetry data to a one-second interval.
3. The System performs a structural validation check on the packet.
4. If the packet is valid, the System extracts the `timestamp`, `sensor_id`, and `value`.
5. The System appends the valid record to a local temporary batch storage.
6. The System repeats this process until the batch reaches the size specified by the Engineer.

Exit Condition: The local batch storage contains the exact number of valid packets required to trigger an analysis.

Exceptions:

- Malformed JSON: The System identifies truncated strings or invalid syntax; it discards the packet and logs a syntax error.
- Schema Errors: The System detects missing mandatory fields (e.g., no `sensor_id`); it discards the packet.
- Type Errors: The System identifies invalid data types (e.g., a string in the `value` field); it ignores the record.

Special Requirements: Filtering must occur immediately upon reception to prevent corrupted data from entering the analysis batch.

Use Case 3: Batch Analysis and Anomaly Reporting

Name: *Analyze Batch and Detect Anomalies*

Participating Actors: Ground Station Engineer

Entry Condition: The local batch file has reached the Engineer-specified size.

Flow of Events:

1. The System reads the accumulated batch of valid telemetry records.
2. The System sorts and prioritizes the monitoring rules (High > Medium > Low).
3. For each timestamp T_n , the System evaluates:
 - Simple Rules: Checks if the absolute value exceeds a fixed threshold.
 - Step Difference Rules: Compares the current value at T_n with the previous value at $T_n - 1$.
 - Stateful Rules: Updates the count of consecutive violations stored in memory.
 - Logical Correlation Rules: Combines Boolean outcomes of multiple rules using AND/OR logic.
4. **Outcome A (Nominal)**: If no rules trigger, the System appends a single aggregated line to `valid_data.csv`.
5. **Outcome B (Anomalous)**: If any rule evaluates to TRUE, the System appends one entry per violation to `alarms.log`, listing all involved sensors and values.
6. The System clears the local batch file to prepare for the next ingestion cycle.

Exit Condition: All records in the batch are processed, and the output files are updated on disk.

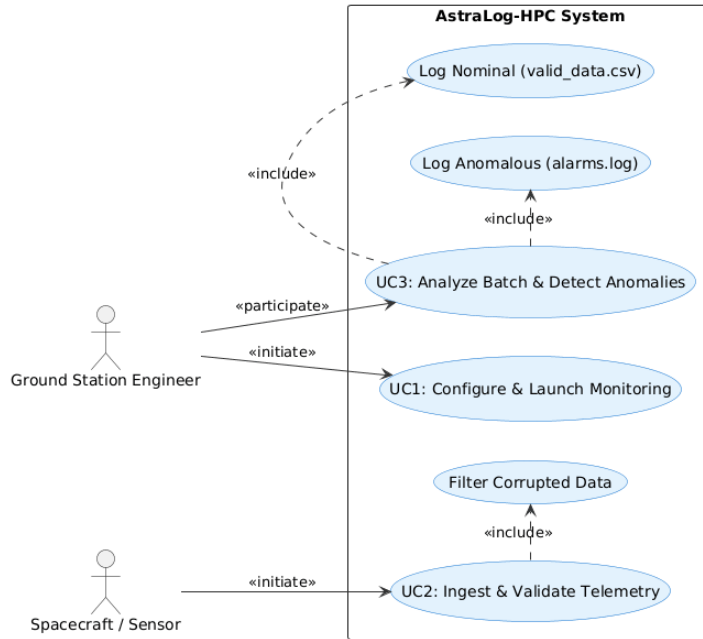


Figure 1: Use Case Diagram of AstraLog-HPC

2.3 Domain assumptions

2.3.1 Configuration and Metadata

- [D.C1] **File Consistency and Integrity:** It is assumed that the configuration files provided (`sensors.yaml` and `rules.json`) are syntactically correct and internally consistent. Every `sensor_id` referenced in a monitoring rule is assumed to exist in the sensor configuration.
- [D.C2] **Parameter Immutability:** The monitoring parameters, including sensor definitions and rule thresholds, are assumed to remain static for the duration of an analysis session.
- [D.C3] **Unique Identification:** Each physical sensor on the spacecraft is assigned a unique `sensor_id` that remains constant throughout the mission's lifetime.
- [D.C4] **Valid Parameter Input:** It is assumed that the operational environment provides a strictly positive integer for the batch size.

2.3.2 Data Nature and Temporal Logic

- [D.T1] **Unified Synchronous System Clock:** The system operates under the assumption of a unified mission clock. While telemetry may be generated at sub-second intervals, time is treated strictly at a 1s granularity. Sub-second offsets are ignored or rounded to the nearest second.
- [D.T2] **Equally Spaced Measurements:** It is assumed that measurements for all sensors are provided at regular, fixed intervals (e.g., exactly one measurement per second).
- [D.T3] **Perfect Data Alignment:** The system assumes that all sensor streams are perfectly synchronized. The n -th measurement for any sensor corresponds to the exact same timestamp as the n -th measurement for all other sensors.
- [D.T4] **Data Continuity:** It is assumed that the spacecraft produces data continuously without gaps or interruptions for the duration of the telemetry batch.
- [D.T5] **Chronological Ordering:** Telemetry records in the input files are assumed to be pre-sorted in strictly increasing chronological order ($T_n \geq T_{n-1}$).

2.3.3 Execution Environment (HPC)

- [D.E1] **HPC Resource Availability:** The Galileo 100 cluster and the SLURM Job Scheduler are assumed to be operational and capable of allocating the requested computational resources (CPUs and memory) for the software.
- [D.E2] **Parallel Processing Support:** The underlying hardware and operating system are assumed to support multi-threading and concurrent execution, enabling the parallel evaluation of monitoring rules.
- [D.E3] **File System Persistence:** The environment provides a reliable file system where the machine has the necessary permissions to read input telemetry and write output reports.

2.3.4 Universal Standards

- [D.U1] **Universal Time Reference:** All timestamps across all telemetry files and configurations use a unified time reference (e.g., UTC) that is consistent across all spacecraft subsystems.
- [D.U2] **Data Formatting:** The provided input telemetry files are assumed to adhere to a standard, consistent CSV format, while the system will strictly generate semicolon-separated output logs (`valid_data.csv` and `alarms.log`) as defined in the mission specifications.

2.4 Requirements

2.4.1 Functional requirements

System Configuration & Initialization

- FR.1 The system shall read and parse a `sensors.yaml` file to identify valid sensor IDs and their associated metadata.
- FR.2 The system shall read and parse a `rules.json` file to establish the monitoring logic (thresholds, counts, and correlations).
- FR.3 The system shall allow the user to specify a batch size representing the total number of telemetry records or the total elapsed time to be processed in a single analysis cycle.

Telemetry Ingestion & Validation

- FR.4 The system shall ingest telemetry records from provided .csv files.
- FR.5 The system shall validate the syntax and schema of each incoming record.
- FR.6 The system shall discard records that contain Malformed JSON (e.g., truncated strings or syntax errors), Schema Errors (e.g., missing mandatory fields), or Type Errors (e.g., non-numerical values in numerical fields) and continue processing the stream.
- FR.7 The system shall accumulate valid telemetry records into a local buffer until the specified batch size is reached.

Monitoring Rule Evaluation

- FR.8 The system shall sort and prioritize rules based on their defined priority level (Low, Medium, High) before evaluation.
- FR.9 Simple Rule: The system shall trigger an alarm if a sensor value v_n violates a static threshold x (e.g., $v_n > x$ or $v_n < x$).
- FR.10 Step-Difference Rule: The system shall trigger an alarm if the absolute difference between the current value v_n and the previous value v_{n-1} exceeds a defined threshold Δ such that: $|v_n - v_{n-1}| > \Delta$.

- FR.11 Stateful Rule: The system shall trigger an alarm only if a specific rule violation persists for N consecutive measurements.
- FR.12 Logical Correlation Rule: The system shall evaluate complex conditions by combining the Boolean outcomes of existing rules using AND or OR logical operators at the same timestamp.

Result Reporting & Logging

- FR.13 The system shall generate a `valid_data.csv` file containing an aggregated summary of every batch where no rule violations were detected.
- FR.14 The system shall generate an `alarms.log` file containing detailed records for every detected anomaly, including: `Timestamp`, `Rule ID`, `Priority Level`, `Violated Sensor ID(s)`, `Current Sensor Value(s)`.
- FR.15 For alarms triggered by Correlation Rules, the system shall explicitly list all sensors and their respective values involved in the parent rules.

2.4.2 Non functional requirements

- NFR.1 Performance & Parallel Efficiency: The system must distribute rule evaluation across multiple threads to achieve a Parallel Efficiency (η) of at least 0.70 (recognized industry standard).
- Formula: $\eta = T1 / (p * Tp)$.
 - Definitions: $T1$ is single-thread execution time; Tp is execution time on p processors.
- NFR.2 Reliability & Robustness: The system must maintain a 0% crash rate. It must successfully discard exactly 100% of corrupted records, specifically Malformed JSON, Schema Errors, and Type Errors, without interrupting valid telemetry processing.
- NFR.3 Verifiability & Formatting: Exactly 100% of output lines must strictly follow the semicolon-separated formats for `valid_data.csv` and `alarms.log`. This strict adherence is mandatory to ensure the software's verifiability.
- NFR.4 Portability & HPC Deployment: The software must be 100% deliverable as a self-contained Singularity container image. This container must execute seamlessly on the Galileo 100 cluster via the SLURM job scheduler.

3 Design

3.1 General Description of the Architecture

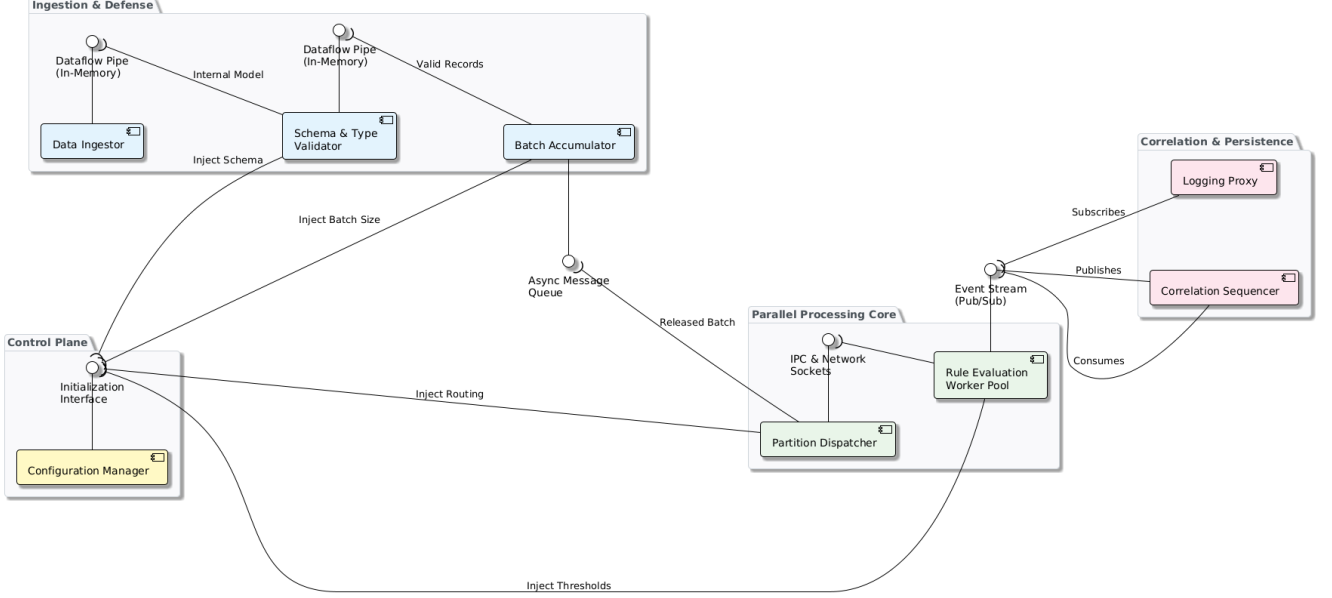


Figure 2: UML Component Diagram of AstraLog-HPC

3.1.1 Architectural Components

- **Data Ingestor:** The primary entry point that reads the telemetry stream via `.csv` files and transforms it into the system's internal data model.
- **Schema & Type Validator:** It filters out syntactically invalid packets, schema errors and type errors before they reach the processing logic.
- **Batch Accumulator:** A buffer component that receives valid telemetry records from the Validator and accumulates them into a local batch file. It holds the data flow until the specified batch size is reached, at which point it releases the complete batch downstream for evaluation.
- **Configuration Manager:** Runs only at startup, loads `sensors.yaml` and `rules.json`, and injects the parsed parameters into the system. Crucially, it sorts all monitoring rules based on their defined priority level (High, Medium, Low) within each rule type before distributing them.
 - **Schema & Type Validator Validator:** receives schema and rules for data validation.
 - **Partition Dispatcher:** receives parameters for deterministic routing.
 - **Rule Evaluation Worker Pool & Correlation Sequencer:** receives thresholds, rule parameters and the pre-sorted execution order.

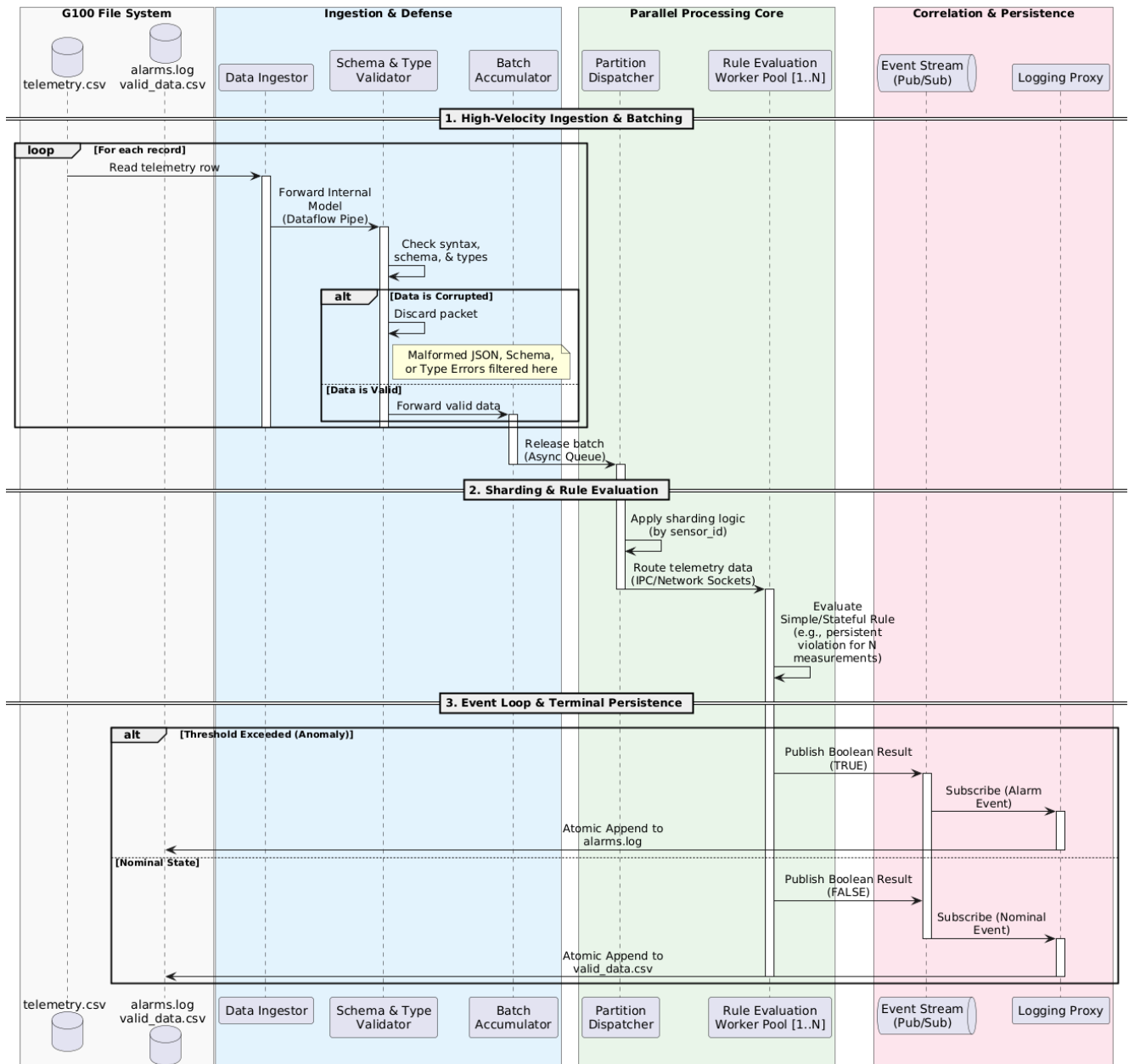
- **Partition Dispatcher:** It shards the batched telemetry stream based on `sensor_id`, ensuring all data for a specific sensor is routed to the same worker.
- **Rule Evaluation Worker Pool:** A pool of concurrent worker instances responsible for executing rule evaluation logic. Each worker processes a dedicated subset of sensors and evaluates Simple, Step Difference, and Stateful rules in the strict priority order defined by the Configuration Manager. For every assigned sensor, a single worker performs the complete rule evaluation pipeline, ensuring consistency and avoiding cross-worker interference. Crucially, each worker maintains an independent internal state, enabling it to track historical measurements and monitor consecutive violations required for stateful rule evaluation.
- **Correlation Sequencer:** A dedicated component that collects Boolean outcomes from the Worker Pool at the same timestamp. It evaluates Logical Correlation rules (also respecting priority order) and publishes the final system state.
- **Logging Proxy:** A centralized component that serializes I/O operations to the parallel file system. It appends healthy states to `valid_data.csv` or violation details to `alarms.log`.

3.1.2 Architectural Connectors

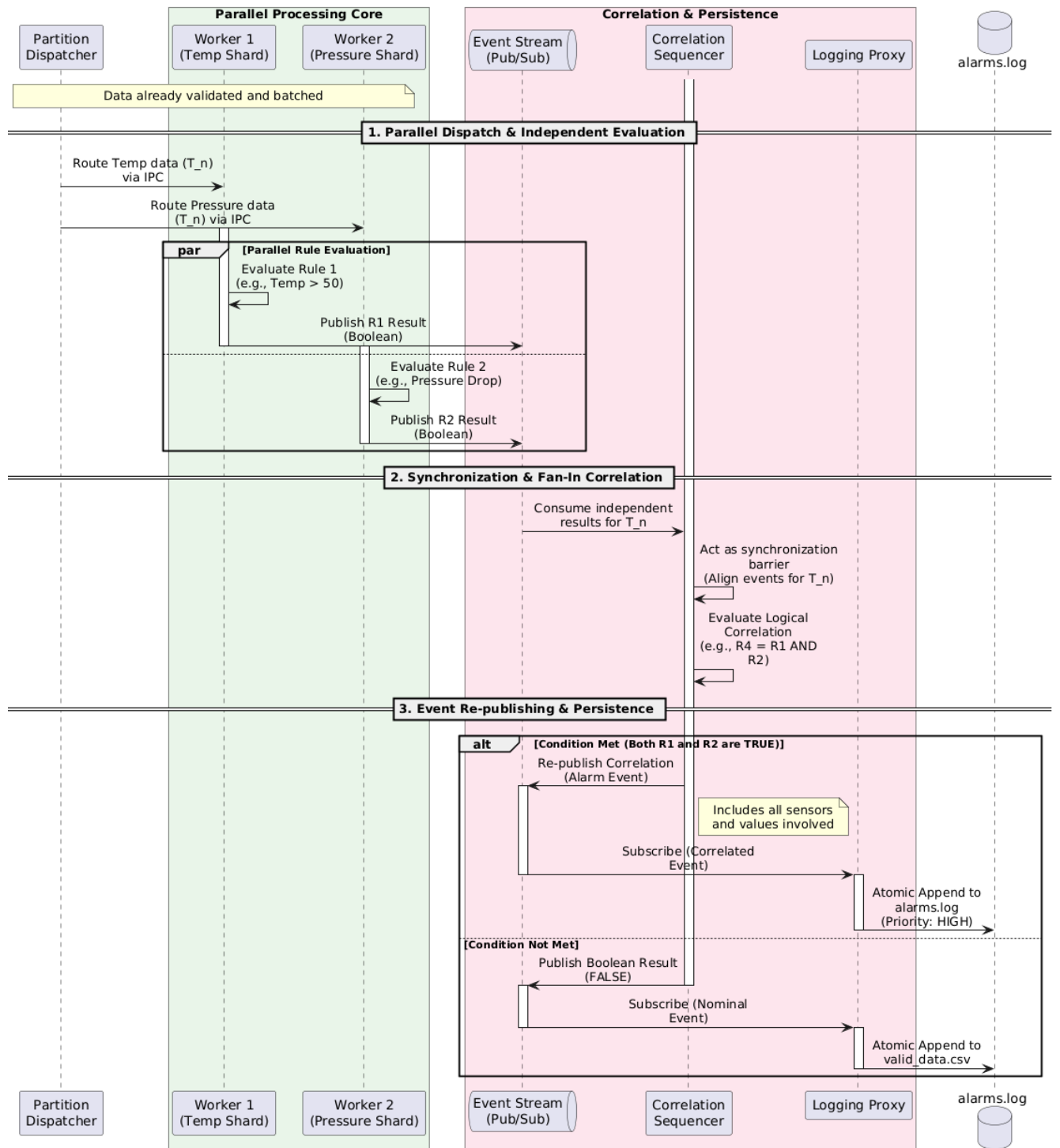
- **Initialization Interface:** A one-to-many bootstrapping connector used by the Configuration Manager to inject static parameters into system components at startup.
- **Dataflow Pipes (In-Memory Streams):** Synchronous connectors employing the Pipe-and-Filter pattern to sequentially integrate the Ingestor, Validator and Accumulator. By operating within a shared execution context, they enable direct information referencing, effectively bypassing the transformation and latency costs typically associated with cross-boundary communication. This establishes a high-throughput defensive boundary, ensuring maximal pre-processing efficiency before telemetry data crosses into the parallelization frontier.
- **Asynchronous Message Queues:** Buffers connecting the Batch Accumulator to the Partition Dispatcher to decouple high-velocity ingestion from downstream processing. These connectors enable task parallelism, allowing the Data Ingestor and Accumulator to continue streaming and batching new telemetry even when the Dispatcher or Worker Pool is occupied.
- **Network Sockets & IPC Channels:** Facilitates a point-to-point, low-latency stream between the Partition Dispatcher and the Rule Evaluation Worker Pool. This connector is a tactical choice to bypass heavy application-layer stacks, ensuring the system achieves High Velocity on the Galileo 100 cluster by minimizing communication overhead.
- **Event Streams (Publish-Subscribe):** An anonymous bus routing Boolean results from Workers and the Sequencer to the Logging Proxy. By avoiding direct component naming, it maximizes decoupling; publishers broadcast data without needing to know the identity or logic of their subscribers. This enables the Correlation Sequencer to act as a second-tier producer, consuming worker results and re-publishing its own findings to the same stream for persistence by the Proxy.

3.2 Sequence diagrams

3.2.1 Simple & Stateful rule evaluation



3.2.2 Logical Correlation Rule Evaluation



3.3 Critical Points and Design Decisions

3.3.1 Maximizing Performance via Divide and Conquer

To satisfy the project requirement of improving rule processing performance through parallelization, we explicitly applied the Divide and Conquer principle. By introducing a

Partition Dispatcher, we shard the massive, batched telemetry dataset into independent sub-problems based on the `sensor_id`. This achieves pure Data Parallelism across the Rule Evaluation Worker Pool, allowing multiple instances to evaluate the rules concurrently and maximizing resource utilization. Furthermore, we optimized the computational load through intelligent Configuration Management. To mitigate I/O bottlenecks, the Configuration Manager centralize metadata parsing at startup, ensuring the parallel file system is accessed only once. Rather than having the Worker Pool waste CPU cycles repeatedly sorting rules for every batch, the Rule Configuration Manager pre-sorts all rules by their defined priority level (High, Medium, Low) before distributing them. This perfectly applies the Separation of Concerns principle, optimizing the workers for pure execution speed.

3.3.2 Mitigating the Limitations of Sharding

While sensor-based partitioning maximizes throughput, it introduces a critical design challenge: a sharding “blind spot.” Because workers only see their assigned shard of sensors, they cannot independently evaluate Logical Correlation Rules, which require analyzing multiple different sensors at the exact same timestamp. To overcome this limitation, the Correlation Sequencer was introduced as a crucial architectural component. It functions as a synchronization barrier at time T_n , performing a “fan-in” of results from all shards to evaluate the logical AND/OR conditions. This mechanism is essential for ensuring temporal causality across the distributed system without breaking the data parallelism of the workers.

3.3.3 Modularity, Cohesion, and Defensive Boundaries

The architecture achieves High Cohesion by enforcing a strict functional separation between the Data Ingestor, the Schema & Type Validator, the Batch Accumulator and the Partition Dispatcher. By isolating these four stages, the system ensures that the specificities of data acquisition, the rigors of integrity enforcement, the logic of batching valid packets, and the mechanics of workload distribution remain completely independent concerns. Crucially, we applied Defensive Design at the absolute boundary of the system. The Schema & Type Validator intercepts malformed JSON, schema errors, and type errors before they can penetrate the pipeline. While centralizing this validation creates a sequential bottleneck at ingestion, we deliberately accepted this trade-off. It guarantees that only “clean” data enters the Rule Evaluation Worker Pool, preventing unhandled exceptions and ensuring that the distributed workers are dedicated exclusively to pure computational logic, entirely decoupled from string-parsing overhead.

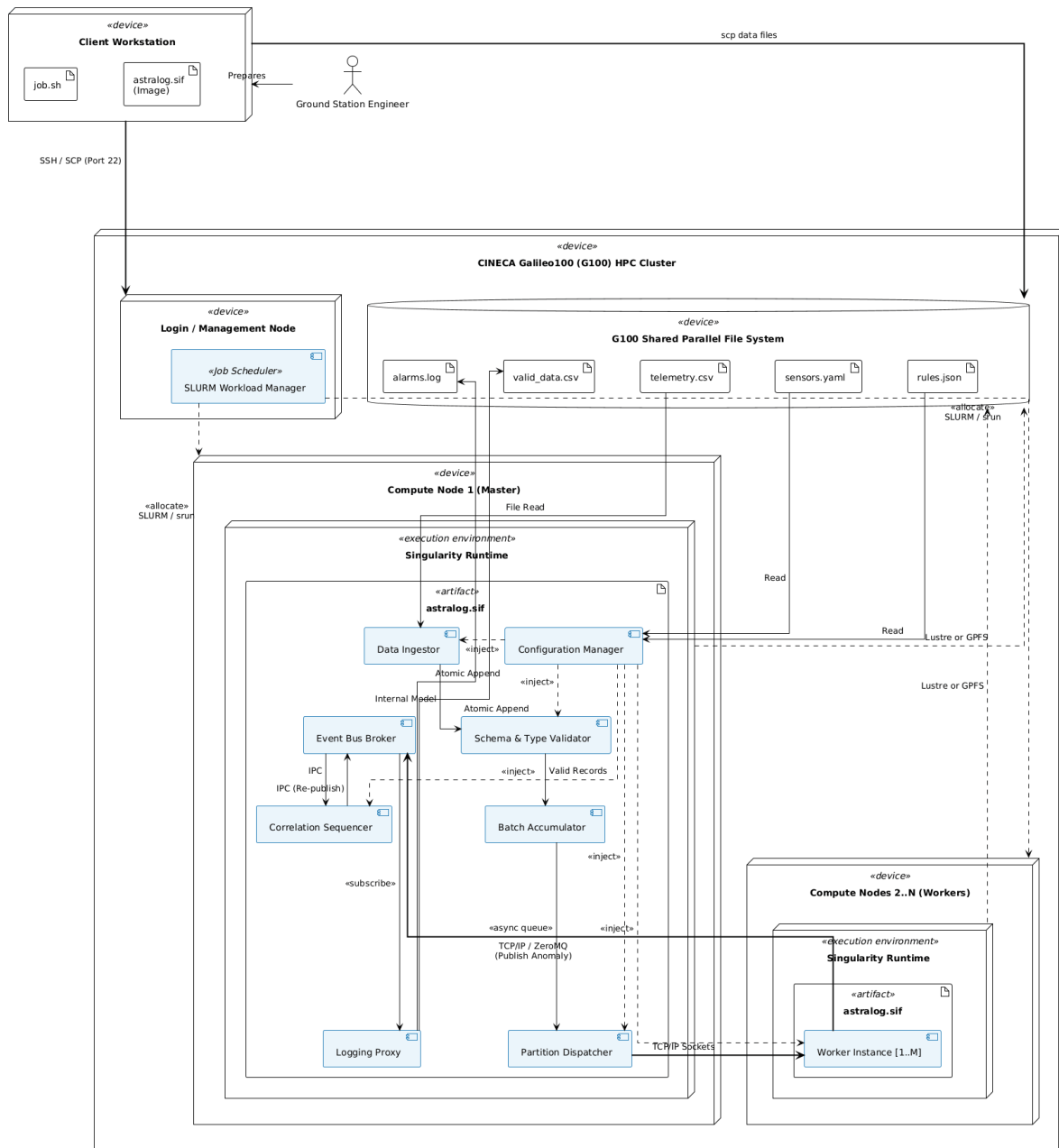
3.3.4 Information Hiding and I/O Bottleneck Management:

To manage the Stateful Rules, which trigger anomalies only if a condition persists for a consecutive number of measurements, we strictly relied on Information Hiding. By housing the measurement persistence logic entirely within the internal state of each worker, we keep the coupling between workers low, as they do not need to share sensitive internal buffers to function. Finally, we addressed the severe risk of performance degradation common in parallel file systems by introducing a Logging Proxy. In a standard Shared Memory architecture, having multiple distributed workers competing for a lock to write to the same file creates a centralized performance bottleneck. By routing all outcomes to

the Logging Proxy, the system serializes the I/O operations in a high-throughput manner, ensuring that the strict formatting requirements for `valid_data.csv` and `alarms.log` are safely maintained without stalling the parallel workers.

3.4 Deployment Diagram

To synthesize these architectural decisions with the physical constraints of the execution environment, the figure below illustrates the deployment topology of the AstraLog-HPC system. The diagram demonstrates how the centralized ingestion and aggregation components (such as the Validator and Logging Proxy) are mapped to the Master Node, while the stateless Rule Evaluation instances are distributed across the compute nodes of the Galileo 100 cluster within isolated Singularity containers.



4 Work distribution and effort

We comprehensively addressed all project phases and analysed the distribution of effort according to the trend illustrated below. By alternating structured collaborative sessions with focused individual work, we ensured consistent coordination and sustained progress throughout the entire project lifecycle.

Phase	Tasks (Hours)	Total
RASD	Actors (3), Use cases (10), Domain assumptions (3), Requirements (5)	21
DD	Architecture (10), Sequence diagrams (3), Design decisions (5)	18
Total		39

4.1 Personal Contribution and Role in Requirements Analysis and System Design

Antonello Anzalone

“My contribution lies in the combination of curiosity and meticulousness that characterizes my approach, allowing me to explore the connection between a practical case study and the formalism of software engineering used to describe situations and represent solutions. In particular, during the requirements analysis phase, I focused on understanding the concrete meaning of the theoretical tools employed, using them to rigorously model every detail of the system based on the specifications defined by the project group. In the design phase, I worked on identifying inconsistencies between the initial ideas and established software design principles, proposing solutions to address edge cases identified both independently and collaboratively within the team. Finally, I coordinated the overall structure and the writing of the final document, ensuring consistency, clarity, and high formal quality.”

Francesco Agosta

"I focused on making sure the design actually worked in practice, not just in theory. I spent a lot of time mapping out the logic through diagrams to keep the team aligned and worked closely across different areas of the project to ensure our choices were technically sound. My main priority was catching those tricky edge cases early on to make sure the final product was solid and reliable. I also stayed focused on the architecture, ensuring it was robust enough to hit all our targets, from functional features to long-term performance."

Gabriele Amodeo

"My primary contribution centered on establishing a comprehensive understanding of the overall problem domain and on the requirements analysis phase. By synthesizing the project's core constraints, I helped shape the high-level architectural design, ensuring our structural decisions aligned perfectly with the demands of a distributed HPC environment."

5 Usage of AI

AI proved to be highly valuable, first and foremost in providing a concrete sense of awareness and depth regarding theoretical and academic tools whose practical usefulness is often not immediately evident.

More specifically, during the requirements analysis phase, AI was used as an additional layer of assurance due to its ability to capture specification details that could easily be overlooked. Its most significant contribution in this phase, however, was its creative support: it helped translate informal and concrete scenarios into highly detailed formal representations, as well as explore more abstract cases with precision. This enabled us to build a clear and well-defined understanding of what the system is and what it is expected to do.

In the design phase, AI was particularly useful in identifying when our initial, rough ideas conflicted with established software design principles. This allowed us to iteratively refine the system architecture, progressively improving our design by leveraging its ability to connect theoretical concepts that initially appeared unrelated but proved to be fundamental when combined.

As a final overarching contribution, AI supported the overall coherence of the project by facilitating alignment between requirements and design choices, ensuring consistency across all stages of development.